CÓDIGOCLARO · MATERIAL DE ESTUDIO

Java: ejercicios resueltos y preparación de examen

Colecciones, objetos, algoritmos pequeños, pruebas y revisión de contratos en Java 21

1. Leer el enunciado y fijar el contrato

Objetivo: Convertir un enunciado de examen en entradas, resultados, restricciones y casos frontera verificables.

Antes de escribir Java, subraya qué datos recibe el método, qué resultado devuelve y cuáles son los límites. Define si una lista nula se permite, si la entrada vacía produce resultado vacío y si el orden importa. Distingue requisito funcional de implementación: el examinador suele evaluar comportamiento observable, no una API concreta salvo que la pida. Después formula ejemplos pequeños y calcula a mano el resultado esperado. El contrato ayuda a elegir tipos, excepciones y estructura de control. Una solución sencilla con casos cubiertos es más fácil de defender que una optimización prematura difícil de verificar.

Código · ejemplo

```
[Java 21]
static int contarMayores(int[] valores, int limite) {
    int total = 0;
    for (int valor : valores) {
        if (valor > limite) total++;
    }
    return total;
}
```

Práctica: Define tres casos de prueba, incluido uno límite, para `contarMayores`.

Respuesta razonada: Con `[]` el resultado esperado es `0`; con `[4, 7, 7]` y límite `7`, es `0` porque la comparación es estricta; con `[8, 7]` es `1`. El enunciado debe decidir qué hacer con `null`, ya que el fragmento lanza `NullPointerException` y no lo maneja explícitamente.

2. Eliminar elementos de una ArrayList

Objetivo: Modificar una lista sin saltar elementos ni invalidar el recorrido durante una eliminación.

Al quitar un elemento de una `ArrayList`, los posteriores desplazan sus índices. Un bucle que incrementa el índice tras eliminar puede saltarse el elemento que ocupaba la posición siguiente. Para borrar por condición, puede recorrerse desde el final hacia el principio, usar `removeIf` o emplear explícitamente un `Iterator` y su operación `remove`. No llames a `list.remove(x)` con un entero sin revisar la sobrecarga: con `List<Integer>`, `remove(int)` elimina por índice y `remove(Object)` elimina por valor. Elige el método que exprese intención y prueba duplicados, primer y último elemento y lista vacía.

Código · ejemplo

```
[Java 21]
List<Integer> values = new ArrayList<>(List.of(2, 4, 4, 7));
values.removeIf(value -> value % 2 == 0);
// values queda [7]
```

Práctica: ¿Cómo eliminarías solo la primera aparición del valor 4 y qué ambigüedad evitarías?

Respuesta razonada: `values.remove(Integer.valueOf(4))` selecciona la sobrecarga que elimina por objeto; `values.remove(4)` intenta eliminar el índice 4. `removeIf` elimina todas las apariciones que cumplen el predicado. La respuesta depende de si el contrato dice primera coincidencia o todos los valores pares.

3. Frecuencia de valores con Map

Objetivo: Contar apariciones con un mapa y comprobar cómo resuelve el problema de claves ausentes.

Un mapa relaciona cada clave con su frecuencia. `Map.merge` puede insertar el primer valor o combinarlo con el anterior en una sola operación. Para frecuencias, una clave de texto se compara con `equals` y `hashCode`; si se crea una clase propia como clave, ambos métodos deben respetar el mismo contrato. Normaliza espacios o mayúsculas solo si el enunciado define que valores equivalentes deben agruparse. No uses una variable global para acumular entre llamadas: crea el mapa dentro del método y devuelve el resultado para mantener la operación repetible.

Código · ejemplo

```
[Java 21]
Map<String, Integer> counts = new HashMap<>();
for (String word : words) {
    counts.merge(word, 1, Integer::sum);
}
```

Práctica: ¿Qué ocurre si `words` contiene `null` y el mapa es `HashMap`?

`HashMap` admite la clave nula y su implementación de `merge` la cuenta bajo esa clave; no lanza `NullPointerException` por ese motivo. `merge` sí exige que el valor nuevo y la función de combinación no sean nulos. Si el dominio no permite palabras nulas, recházalas o normalízalas explícitamente antes de contarlas.

4. Orden natural y Comparator

Objetivo: Definir ordenación coherente y distinguir comparación natural de una regla específica de presentación.

Una clase implementa `Comparable<T>` cuando tiene un orden natural único y documentado. `compareTo` devuelve negativo, cero o positivo; no debe depender de restar enteros porque la resta puede desbordar. `Comparator` permite varias reglas externas sin imponer una sola al objeto, por ejemplo ordenar personas por apellido y después por nombre. Para ordenar con estabilidad semántica, añade un criterio de desempate si los valores principales empatan. La igualdad entre `compareTo` y `equals` es muy recomendable para estructuras ordenadas, aunque Java no la impone en todos los contextos. Prueba repetidos, extremos numéricos y valores nulos según contrato.

Código · ejemplo

```
[Java 21]
Comparator<Person> byName =
    Comparator.comparing(Person::lastName)
              .thenComparing(Person::firstName);
```

Práctica: ¿Por qué `Integer.compare(a.age(), b.age())` es preferible a `a.age() - b.age()`?

Respuesta razonada: `Integer.compare` no puede desbordarse por una resta intermedia y expresa directamente el resultado requerido. La resta puede producir signo incorrecto para edades u otros enteros extremos. Si la edad es el único criterio, dos personas pueden empatar; añade un desempate si el orden total de salida debe ser reproducible.

5. Encapsulación e invariantes

Objetivo: Modelar un objeto que proteja sus reglas en vez de exponer campos modificables libremente.

La encapsulación mantiene el estado interno detrás de una interfaz y permite verificar invariantes en cada cambio. Un objeto `Cuenta` puede impedir un saldo negativo en una retirada, o devolver un resultado que indique fondos insuficientes. Los campos suelen ser privados; la clase ofrece operaciones con nombres que reflejan el dominio, no setters genéricos que permitan cualquier transición. Las copias defensivas protegen colecciones mutables. Define la unidad monetaria y usa `BigDecimal` con escala y política de redondeo acordadas para dinero; un `double` representa aproximaciones binarias. Prueba el estado antes y después de cada operación inválida.

Código · ejemplo

```
[Java 21]
final class Cuenta {
    private BigDecimal saldo;
    Cuenta(BigDecimal inicial) { this.saldo = inicial; }
    BigDecimal saldo() { return saldo; }
}
```

Práctica: ¿Qué regla falta para que esta clase proteja realmente el saldo?

Respuesta razonada: El constructor debe validar que `inicial` no sea nulo y cumpla la regla de saldo; además, las operaciones de ingreso y retirada deben validar importes y preservar la invariante. Un getter no debe permitir mutar el objeto devuelto si el estado fuera una colección. La encapsulación solo ayuda cuando los caminos de escritura están controlados.

6. Composición e interfaces

Objetivo: Reducir acoplamiento usando contratos pequeños y colaboradores inyectados de forma explícita.

La composición delega una tarea a otro objeto; una interfaz permite programar contra un contrato y sustituir la implementación en pruebas o en otros entornos. No todo dato requiere una interfaz: abstrae cuando existe una variación real o una dependencia externa que conviene aislar. Una clase debe conservar una sola responsabilidad coherente. Si crea dentro del método su propio acceso a reloj, red o base de datos, puede ser difícil probar resultados. Inyectar el colaborador por constructor hace visible la dependencia. Evita jerarquías de herencia profundas cuando una delegación simple expresa mejor el comportamiento.

Código · ejemplo

```
[Java 21]
interface Clock { Instant now(); }
final class ReportService {
    private final Clock clock;
    ReportService(Clock clock) { this.clock = clock; }
}
```

Práctica: ¿Cómo probarías que un informe recibe la fecha correcta sin depender del reloj real?

Respuesta razonada: Sustituye `Clock` por una implementación fija que devuelva un instante conocido y verifica el contenido generado. Así la prueba es determinista y no falla al cruzar medianoche o cambiar la zona horaria. En producción, la implementación real puede delegar en `Instant.now()`.

7. Excepciones y validación de entrada

Objetivo: Diferenciar entrada inválida, ausencia de resultado y fallo inesperado sin ocultar excepciones.

Valida los datos en el límite donde entran al dominio. Usa excepciones para condiciones excepcionales y resultados explícitos para alternativas habituales que la persona que llama debe manejar. No captures `Exception` para continuar con valores por defecto silenciosos. Una excepción debe comunicar qué dato falló sin incluir secretos. Las excepciones verificadas obligan a declararlas o capturarlas; las no verificadas suelen representar errores de programación o argumentos inválidos según el contrato. Traduce errores solo cuando puedas añadir contexto o recuperación. Prueba tanto el camino válido como el tipo de fallo que el enunciado espera.

Código · ejemplo

```
[Java 21]
static int parsePositive(String text) {
    int value = Integer.parseInt(text);
    if (value <= 0) throw new IllegalArgumentException("Debe ser positivo");
    return value;
}
```

Práctica: ¿Qué entradas producen `NumberFormatException` y cuáles `IllegalArgumentException`?

Respuesta razonada: Una cadena no convertible, como `"abc"`, produce `NumberFormatException`, que hereda de `IllegalArgumentException`. Una cadena válida que representa cero o un entero negativo produce la excepción explícita del método. Si la interfaz necesita diferenciar mensajes, captura con cuidado y traduce a un error de dominio documentado.

8. Ficheros y recursos

Objetivo: Leer texto con codificación definida y asegurar el cierre de recursos incluso ante fallos.

La API `Files` trabaja con `Path` y ofrece métodos de lectura y escritura. Fija UTF-8 si el formato lo requiere; depender de la codificación predeterminada de la máquina puede cambiar entre entornos. `Files.readString` resulta cómodo para archivos pequeños, pero no debe cargar en memoria un archivo sin límite conocido. En procesamiento grande, recorre líneas o usa un flujo con cierre automático. La ruta debe provenir de una política de confianza y no permitir escapar del directorio asignado. Distingue errores de formato, permisos y ausencia para devolver un mensaje útil sin revelar rutas sensibles.

Código · ejemplo

```
[Java 21]
Path file = Path.of("datos.txt");
String text = Files.readString(file, StandardCharsets.UTF_8);
```

Práctica: ¿Por qué no usarías `readString` para un archivo de tamaño ilimitado proporcionado por un usuario?

Respuesta razonada: El método carga el contenido completo en memoria, por lo que un archivo enorme puede agotar recursos. Se debe limitar tamaño y utilizar lectura en streaming. También hay que validar que la ruta esté dentro del área permitida, comprobar permisos y gestionar `IOException` en el punto donde se pueda explicar o recuperar el fallo.

9. Pruebas unitarias con JUnit

Objetivo: Escribir una prueba pequeña que compruebe el contrato y los límites de un método Java.

Una prueba unitaria fija una entrada y verifica una expectativa observable. JUnit 5 permite marcar métodos con `@Test` y usar aserciones como `assertEquals` o `assertThrows`. El nombre debe describir el escenario y el resultado, no el orden interno de llamadas. Una prueba no es más fuerte por comprobar cada línea; debe detectar errores plausibles y representar reglas de negocio. Separa datos de prueba, ejecuta cada caso de forma independiente y evita depender del reloj, red o filesystem real salvo que sea una prueba de integración. Si cambia la implementación y la prueba se rompe aunque el contrato siga igual, probablemente se acopló a detalles internos.

Código · ejemplo

```
[JUnit 5]
@Test
void cuentaMayoresExcluyeElLimite() {
    assertEquals(1, contarMayores(new int[]{8, 7}, 7));
}
```

Práctica: Añade una prueba para el array vacío y explica la expectativa.

Respuesta razonada: La expectativa es cero, porque no existe ningún elemento mayor que el límite. Esa prueba detecta inicializaciones incorrectas y accesos al primer índice sin validar longitud. Añade también un caso con un valor igual al límite para confirmar que el enunciado usa `>` y no `>=`.

10. Simulacro de examen y revisión final

Objetivo: Resolver una tarea de programación con tiempo limitado, justificar decisiones y revisar casos de error.

En un examen, dedica los primeros minutos a convertir requisitos en una firma y ejemplos. Implementa primero el camino principal; después añade validaciones y pruebas para límites. Compila con el nivel de Java establecido por el centro, pues APIs recientes pueden no estar disponibles en Java 8 o 11. Revisa nombres, tipos, acceso a estado, contrato de retorno y complejidad. No copies una API de memoria sin verificar que existe en la versión objetivo. Deja tiempo para ejecutar, leer errores de compilación y corregir uno por uno. Explica decisiones clave con frases cortas: qué estructura elegiste, qué caso cubre y por qué.

Código · ejemplo

```
[Java 21]
static Optional<String> primeroMayor(List<String> valores, int minimo) {
    return valores.stream().filter(s -> s.length() > minimo).findFirst();
}
```

Práctica: ¿Qué significa que `findFirst` dependa del orden de la lista y qué debes especificar?

Respuesta razonada: El método devuelve la primera coincidencia según el orden de encuentro. Si la colección no tiene orden definido, el resultado puede variar. Para un resultado determinista, usa una lista ordenada o define un `Comparator`; además, especifica si lista nula y elementos nulos se admiten y qué significa “primero”.

Fuentes oficiales

Oracle · Java Language Specification, Java SE 21 · Dev.java · Object-Oriented Programming · Oracle · Java SE 21 Collections API · Oracle · Java SE 21 Files API · JUnit 5.14.4 · User Guide